プログラミング言語実験 (C++)
第 5 回: プログラム作成のまとめ，画像処理，パターン認識の基礎と演習

市野　将嗣

第 5 回の実験では，プログラム作成のまとめ，画像処理，パターン認識の基礎について学ぶ.

1　プログラム作成のまとめ

研究でプログラミングを行っていくと，ソースコードの長いプログラムを作成することが多くなる．その際，main 関数にすべての処理を書くのは避けるのが良い．処理が多くなるとプログラムが長くなり，それにつれてバクが生じる可能性も高くなる.

この実験で，C++の基礎を扱いました．機能ごとに関数にまとめて，必要な時に手軽に使えるようにしておくことは重要です (いちいち始めから処理内容をプログラミングする手間が省けます，さらにバグも抑えられるかもしれません)．さらに，クラスの機能をうまく使うことで，過去に作成したプログラムを効果的に利用することができます．前回も言いましたが，カプセル化を使うことで，予期しない入力値をチェックすることができます．合わせて，変数の名前の付け方には注意してください．内容を表す名前が良いでしょう．クラスには，この実験で触れていない機能もあります．各自で勉強し，効果的にプログラミングを行うことができるように経験を積んでもらえればと思います.

2　パターン認識の基礎

パターン認識とは，観測されたパターンをあらかじめ定められた複数のクラス (もしくはカテゴリとも呼ぶ，C++でのクラスとは異なるので注意) のうちの 1 つに対応させる処理のことである．例えば，バイオメトリクスであれば，入力された指紋データはだれの指紋であるかを判定する処理ということになる．観測されたパターンとは，顔や指紋のような画像であることもあれば，音声のような時系列情報であることもある．もちろん，画像や音声のみならず様々なセンサから取得されるデータを認識することもパターン認識という.

入力されたデータをクラスに対応させる機械のことを識別器と呼んでいる．パターン認識を用いた識別器は，図 1 のように前処理，特徴抽出，識別で構成される．パターンが入力されるとまず，前処理でノイズ除去，正規化などの処理を行う．続いて，特徴抽出で，膨大な情報を持つ原パターンから識別に必要な本質的な特徴のみを抽出する．この特徴をもとに識別では，識別処理を行う．識別では，入力パターンに対して複数のクラスのうち

の１つを対応させる．そのため，あらかじめシステムに登録しておく学習データを用意して，その学習データを用いてモデルを作成しておく．そして入力されたテストデータから抽出された特徴を事前に用意したモデルと照合することにより入力パターンの所属するクラスを出力する．パターンが入力されてから出力されるまでの前処理，特徴抽出，識別をまとめて認識と呼ぶ．

学習時に学習パターンがその所属クラス名とともに与えられる学習法を教師付き学習，学習パターンにクラスのラベルが付いていない場合の学習を教師なし学習と呼ぶ．

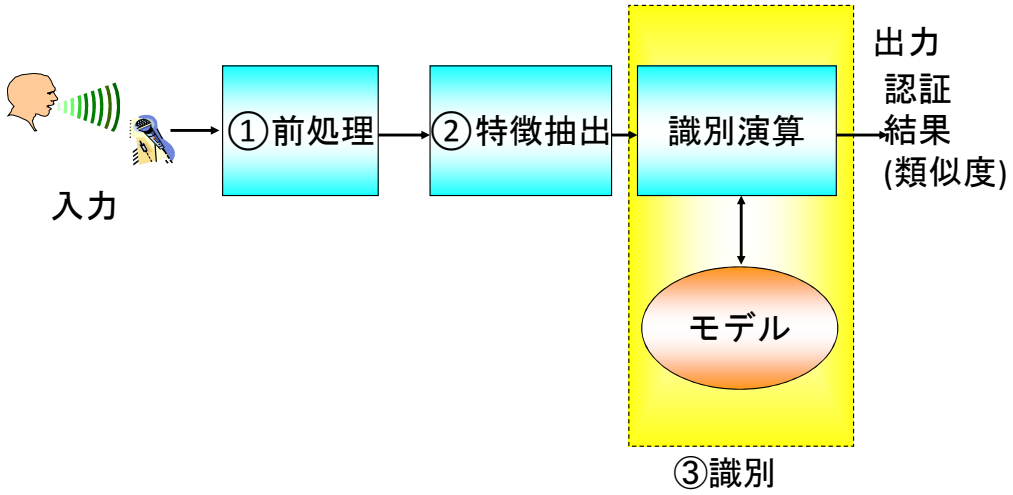

図 1: 識別器の構成

3　機械学習

既に多くの機械学習アルゴリズムが提案されています．その中でも，今回の実験では，機械学習の代表的な手法であるサポートベクターマシン (SVM) を利用する．SVM がどのような手法であるのかについて以下，概要を説明する．

3.1　SVM の概要

図 2 に示すように，●サンプルのクラスと▲サンプルのクラスを分離する識別境界面を求める問題を考えます．2 つのクラスを分離する識別境界面はいろいろ引けますが，SVM は，識別境界面とその境界面に一番近いサンプルとの距離 (マージンと呼びます) が最大になる識別境界面を求めるアルゴリズムです．

一般的な認識の問題では，識別境界面を作成するための学習データとその識別境界面を利用した識別精度を評価するためのテストデータは，同一のものを用いません．学習データをすべて正しく識別できたとしても，テストデータも正しく識別できるとは限りません．マージンが大きい識別境界面を作成できれば，学習データと少し異なるテストデータでも正しく識別できることが期待できます．認識では，これを汎化性能と呼びます．SVM は，この汎化性能を高めるためにマージンを最大化する識別境界面を求めます．

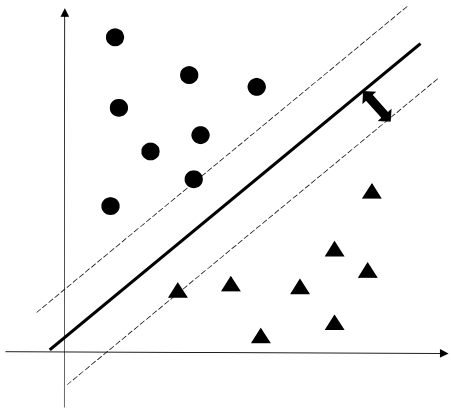

図 2: 学習データと識別境界面

3.1.1 マージンを最大にする識別境界面

サンプルとそれに対するクラスラベルがついた学習データを $(x_i, y_i), i = 1, \cdots, N$,$y_i = 1$ または $y_i = -1$ とする．ただし，y_i はクラスラベルで，正例を 1，負例を -1 とする．このとき，特徴空間上で識別面は

$$\omega \cdot x + \omega_0 = 0 \tag{1}$$

で表現できる．すると，この境界面と i 番目のサンプル x_i との距離 $D(x_i)$ は，点と直線の距離の公式を用いて

$$D(x_i) = \frac{|\omega \cdot x + \omega_0|}{||\omega||} \tag{2}$$

と表すことができる．さらに，学習パターンと識別面との最小距離は,

$$\min_{i=1,\cdots,N} D(x_i) = \frac{1}{||\omega||} \tag{3}$$

で表すことができる．これより，マージンを最大化する識別境界面を求めるには，$||\omega||$ を最小化する必要があります．通常は解析しやすいように,

$$\min = \frac{1}{2}||\omega||^2 \tag{4}$$

という問題として扱います．このとき，識別面としてすべての学習データを識別できるという以下の条件を加えます.

$$y_i(\omega \cdot x_i + \omega_0) \geq 1, i = 1, \cdots N \tag{5}$$

この制約付き最適化問題をラグランジュの未定乗数法を用いて解くと，ラグランジュ乗数 α_i として,

$$\omega = \sum_{i=1}^{N} \alpha_i y_i x_i \tag{6}$$

と求めることができる．

ω_0 については，

$$\omega_0 = -\omega \cdot x \tag{7}$$

に正例，負例に属するサポートベクトルを代入しその平均を用いる．これで，マージンを最大にする識別境界面を求めることができる．

3.1.2 ソフトマージンによる識別境界面

各クラスのデータ領域が重なっている場合は，各クラスのサンプルが入り混じることがある．この時には，全ての学習データを正しく識別することはできない．それを踏まえ，全ての学習データを正しく識別できるようにするのではなく，図 3 のようにいくつかのサンプルの識別誤りは許容しながら識別境界面を求めるという考え方があります．

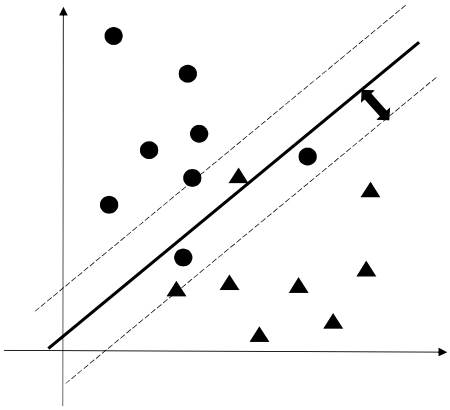

図 3: 学習データとソフトマージンによる識別境界面

式 (5) がすべての学習データを正しく識別できる条件であるので，この制約を弱めることを考えます．具体的には，変数 ξ_i を用いて，i 番目のデータが制約を満たしていない程度を表します．ξ_i は，図 2 のマージンを構成する点線の内側に向かって距離 ξ_i だけデータが入りこむことを意味し，制約を満たさない程度ですので，小さいほうが望ましい．すると，上記の 3.1.1 の問題に ξ_i を加えて，

$$\min = \frac{1}{2}||\omega||^2 + C\sum_{i=1}^{N}\xi_i \tag{8}$$

と表すことができる．このとき，以下の制約条件を加えます．

$$y_i(\omega \cdot x_i + \omega_0) \geq 1 - \xi, i = 1, \cdots N \tag{9}$$

ここで，C は制約を満たさないデータをどの程度の重みで組み込むかを決める定数を表す[1]．なるべく小さい ξ_i が選ばれるように，定数 C を用いた制約付き最適化問題をラグラ

[1] C は分類の正答率などを見ながら，自分で調整を行います．

ンジュの未定乗数法で解くと，

$$\omega = \sum_{i=1}^{N} \alpha_i y_i x_i \tag{10}$$

と求めることができる．ただし，$0 \leq \alpha_i \leq C$ を満たす必要がある．

ω_0 については，

$$\omega_0 = -\omega \cdot x \tag{11}$$

に正例，負例に属するサポートベクトルを代入しその平均を用いる．これで，ソフトマージンによる識別境界面を求めることができる．C はマージンよりも識別境界側に出てしまう学習データに対する損失の重みに対応する．C を大きくすると，この損失を重く見積もることになり，3.1.1 節で説明した SVM(ハードマージンの SVM) に近づく．C を小さくしていくと，マージンよりも識別境界側のデータを数多く許容するようになる．

3.1.3 非線形の SVM

3.1.1 節，3.1.2 節で説明した SVM では，線形の識別境界面しか求めることができません．認識の問題では，線形の識別境界面では正しく分離できないことも多く，非線形の識別境界面を求めることが必要となります．非線形識別面を求めることができるようにカーネル関数を用いて SVM を拡張することが提案されています．ここでは，概要を説明します．

カーネル関数を用いる基本的なアイディアは，元の特徴空間をより高次元の空間に写像し，その空間で線形のアルゴリズムを用いて識別境界面を求めると，元の特徴空間では非線形識別面を求めたことになる，というものです．ただし，むやみに次元数を上げるのではなく，もとの特徴空間に対して，もとの特徴空間におけるデータ間の距離関係を保存する方式で高次元に非線形変換します．

カーネル関数を $K(x, x')$，非線形写像を ϕ としたとき，

$$K(x, x') = \phi(x)^t \phi(x) \tag{12}$$

を仮定する．すると，写像後の空間での識別関数は，

$$g(\phi(x)) = \omega^t \phi(x) + \omega_0 = \sum_{i=1}^{N} \alpha_i y_i K(x, x_i) + \omega_0 \tag{13}$$

と表せる．識別関数を求める際，高次元空間での内積を求める必要があるが，その内積の計算をカーネル関数を計算すればよい，という形で表現できることを意味している．

良く使うカーネル関数としては，ガウシアンカーネル (RBF カーネルともいう) がある．

$$K(x, x') = \exp\left(\frac{-||x - x'||^2}{\sigma^2} \right) \tag{14}$$

そのほか，多項式カーネルがある．

$$K(x, x') = (x^t x' + 1)^p \tag{15}$$

SVM を利用して認識する場合には，この σ や p の値をいろいろと変えて認識を行い，一番認識精度が良い値に設定する．

4　OpenCV の利用

今回の実験では，OpenCV を用いた認識の実験を行う．OpenCV は，コンピュータビジョンと機械学習のライブラリです．様々な関数が用意されていますので，これらを使いこなすといろいろなことができます．例えば，画像認識，理解の研究やバイオメトリクスなどのセキュリティの研究でも利用されています．OpenCV は，オープンソースであり，無料で利用できます．プログラミング言語実験において既に，C#の実験で利用していると思いますが，基本的なところをもう一度確認しておきます．今回の実験では，IED の Linux 環境にインストールされている 4.2.0 を利用してもらいます．OpenCV を使ってプログラミングすることを経験してもらいたいと思っています．詳細の機能を確認したい場合は，実験用の Web ページに OpenCV4.2.0 のマニュアルページへのリンクを張っているので，アクセスしてほしい．

4.1　Visual Studio の設定 (2020 年度まで)

IED には OpenCV がインストールされている．VisualStudio にて OpenCV を利用する場合はファイルやディレクトリを設定する必要があります．今年度は VisualStudio を利用しないので省略．

4.2　Linux 環境での実行 (2021 年度以降)

IED の Linux には OpenCV4.2.0 がインストールされている．OpenCV を利用したソースコードをコンパイルするコマンドは下記となる．

```
g++ foo.cpp -o foo.out ‘pkg-config --libs --cflags opencv4‘
```

C++のプログラムファイル `foo.cpp` から実行型ファイル `foo.out` が作成される．
実行するには下記のコマンドを実行する．

```
./foo.out
```

引数を取るプログラムの場合には，引数も併記する．

```
./foo.out sample.txt
```

4.3　配列

OpenCV では，配列を cv::Mat を利用して表します．配列オブジェクトのメンバーとしては，行と列の数を示す rows と cols，次元数を示す dim，配列データが格納されている場所へのポインタ data などがあります．配列データが配列オブジェクトそのものに格納

されているわけではなく，cv::Mat オブジェクトは，データ領域を示すヘッダと考えるとわかりやすいと思います.

プログラム例を下記に示します.

プログラムコード

```
 1 #include <iostream>
 2 #include <opencv2/opencv.hpp>
 3
 4 int main()
 5 {
 6     // 3x3 単位行列 (= cv::Mat m(3,3, CV_64F) )
 7     cv::Mat m = (cv::Mat_<double>(3, 3) << 1, 0, 0, 0, 1, 0, 0, 0, 1);
 8
 9     std::cout << "m=" << m << "\n" << "\n";
10
11     // 行列の演算
12     cv::Mat A = (cv::Mat_<double>(3, 3) << 1, 3, 2, 2, 3, 1, 4, 1, 2);
13     cv::Mat B = (cv::Mat_<double>(3, 3) << 5, 1, 2, 4, 1, 8, 9, 3, 1);
14     std::cout << "A=" << A << "\n\n";
15     std::cout << "B=" << B << "\n\n";
16
17     std::cout << "A+B=" << A+B << "\n";
18
19     return 0;
20 }
```

実行結果

```
m=[1, 0, 0;
 0, 1, 0;
 0, 0, 1]

A=[1, 3, 2;
 2, 3, 1;
 4, 1, 2]

B=[5, 1, 2;
 4, 1, 8;
 9, 3, 1]

A+B=[6, 4, 4;
 6, 4, 9;
```

```
13, 4, 3]
続行するには何かキーを押してください . . .
```

名前空間の宣言については，std と cv の違いを見せるためにしていません．宣言しても問題ありません．また，CV_64F は Mat 型変数の画素の型の種類を示す名前であり，メンバー関数 type() の戻り値となる．

4.4 画像の読み込み

画像の読み込みには imread を使います．処理結果を書き込みには，imwrite を使います．プログラム例を下記に示します．

プログラムコード

```
#include<iostream>
#include<opencv2/opencv.hpp>
using namespace std;
using namespace cv;
string win_src = "source";
string win_dst = "destination";

int main(int argc, char **argv)
{
    string file_src = "logo.png"; // 入力画像のファイル名
    string file_dst = "out.png"; // 出力画像のファイル名
    Mat img_src = imread(file_src, 1); // 入力画像の読み込み

    Mat img_dst;

    //画像の読み込み確認
    if (!img_src.data) {
        cout << "error" << endl;
        return -1;
    }

    // ウインドウ生成
    namedWindow(win_src, WINDOW_AUTOSIZE);
    namedWindow(win_dst, WINDOW_AUTOSIZE);

    // ここに処理を記述する (blur を使って画像を平滑化している)
    blur(img_src, img_dst, Size(5, 5));

```

```
    imshow(win_src, img_src);  // 入力画像を表示
    imshow(win_dst, img_dst); // 出力画像を表示
    imwrite(file_dst, img_dst); // 処理結果の保存

    waitKey(0); // キー入力待ち
    return 0;
}
```

入力画像

図 4: 入力画像

出力画像 (平滑化されていることがわかる)

図 5: 出力画像

4.5 csv ファイルの読み込み

OpenCV には，csv ファイルを読み込む関数 loadFromCSV があります．プログラム例を以下に示す．下記では，loadFromCSV を使って cv::ml::TrainData オブジェクト raw_data を生成し，cv::Mat 型の行列 data にデータを取り出しています．

プログラムコード

```
 1 #include <iostream>
 2 #include <opencv2/opencv.hpp>
 3
 4 int main()
 5 {
 6     // csv データの読み込み
 7     cv::Ptr<cv::ml::TrainData> raw_data = cv::ml::TrainData::loadFromCSV
 ("file.csv", 0,-2,0);
 8
 9     // データの切り出し
10     cv::Mat data(3, 2, CV_64FC1);
11     data = raw_data->getSamples();
12
13     std::cout << data << std::endl;
14
15     return 0;
16 }
```

file.csv の中身

```
3.4,2.2
5.1,3.3
6.3,5.6
```

実行結果

```
[3.4000001, 2.2;
 5.0999999, 3.3;
 6.3000002, 5.5999999]
続行するには何かキーを押してください . . .
```

4.6 SVM の実装

詳細の説明は OpenCV の web ページ (実験ホームページにリンクを張っている) を見てもらいたいが，基本的なところをここで説明する．

SVM による識別器を生成するためには，cv::ml::SVM クラスのメンバ関数 create を用いる．コードとしては例えば，下記のように記載する．

```
Ptr<cv::ml::SVM> svm = cv::ml::SVM::create();
```

SVM の学習の際，パラメータを設定する必要がある．パラメータを設定するには，以下に示すような関数を利用する．

- cv::ml::SVM::setType(int val);

  SVM の型を指定できる．型としては，C_SVC，NU_SVC，ONE_CLASS，EPS_SVR，NU_SVR を指摘できる．デフォルトは C_SVC である．上記で説明したソフトマージン SVM を利用する際は，下記のように C_SVC を指定する．

  ```
  SVM->setType(cv::ml::SVM::C_SVC);
  ```

- cv::ml::SVM::setC(double val);

  引数 val に上記の説明であったパラメータ C の値を設定する．デフォルトは 0 となる．C に 0.1 を設定する際は，下記のように記載する．

  ```
  svm->setC(0.1);
  ```

- cv::ml::SVM::setKernel(int kerneltype)

  SVM のカーネル関数を指定できます．SVM::LINEAR を指定すると線形カーネルを使います．線形の識別面を作成します．SVM::RBF を指定するとガウシアンカーネルを使います．非線形の識別面を作成します．SVM::POLY を指定すると多項式カーネルを使います．そのほか，SVM::SIGMOID，SVM::CHI2，SVM::INTER などがあります．

  線形カーネルを使用する際には下記のように指定する．

  ```
  SVM->setKernel(cv::ml::SVM::LINEAR);
  ```

- cv::ml::setTermCriteria(const cv::TermCriteria & val)

  SVM の学習のための繰り返し計算の終了条件を指定します．最大繰り返し回数やパラメータの許容更新量などを指定します．例えば，繰り返し回数を 1000 回，パラメータの許容更新量を 1e-6 とする場合は下記のように指定する．

  ```
  SVM->setTermCriteria(TermCriteria(TermCriteria::MAX_ITER, 1000, 1e-6));
  ```

SVM による識別器を生成して，パラメータ設定をした後，識別器の学習を行います．学習を行うには，cv::ml::StatModel クラスのメンバ関数である train(InputArray samples, int layout, InputArray responses) を使います．samples には，学習データが保存されている配列を指定します．layout には，1 つのデータが行方向 (ROW_SAMPLE) と列方向 (COL_SAMPLE) のどちらに並んでいるかを指定します．responses には，学習データに対応するラベルが保存されている 1 次元配列を指定します．例えば，下記のように指定する．

```
svm->train(trainingDataMat, ROW_SAMPLE, labelsMat);
```

SVM の学習がすんだら，SVM 識別器でテストデータをそれぞれどのクラスデータかを識別させます．関数としては，cv::ml::StatModel クラスのメンバ関数である predict(InputArray samples, OutputArray results, int flags) を使います．samples には，テストデータを指定します．results には，識別結果を保存する行列を指定できます．flags には，cv::ml::StatModel::Flags で定義されている変数を指定できます．

例えば，下記のコードでは，sampleMat の識別結果ラベルを response に代入している．

```
float response = svm->predict(sampleMat);
```

始めにも説明したように，認識には学習フェーズと識別フェーズがあります．SVM による識別器を作成・学習し，データを識別する際の手順をまとめると下記となる．

学習フェーズ:

1. 学習データを読み込む
2. 学習データのラベルを作成する
3. SVM 識別器を作成する
4. SVM のパラメータを設定する
5. 学習データを用いて SVM を学習する

識別フェーズ:

1. テストデータを読み込む
2. SVM による識別器でテストデータのラベル値を予測する
3. SVM の予測ラベルと正解ラベルを比較し，正しく識別されているかをチェックする．

学習データとそれに対応するラベルを SVM->train の引数に与えて，正例と負例を識別する識別器を学習します．つまり，学習データ (ベクトルデータ) とラベルを対にして学習させます．各ラベルに対応した学習データが適切に分けられるように識別境界を求めることになります．そのために学習データの読み込みやラベルの作成を行います．識別では，テストデータを作成した識別器に入力し，予測クラス (正例か負例) を出力します．

SVM を実装したプログラムコードと学習データ，テストデータ，実行結果の例を以下に示します．この例は，正例 (ラベルは “1”) と負例 (ラベルは “-1”) の 2 クラスの 2 次元ベクトルの学習データを用いて SVM を学習し，テストデータそれぞれを SVM で正例と負例どちらであるかを判定しています．

学習データ train.csv の中身 (2 次元ベクトルデータ，1 行目が正例，2 から 4 行目が負例)

```
501,10
255,10
501,255
10,501
```

テストデータ test.csv の中身 (2 次元ベクトルデータ)

```
255,255
490,15
```

プログラムコード

```
#include <iostream>
#include <fstream>
#include <opencv2/opencv.hpp>
using namespace std;

int main()
{

    /*-----------学習データの読み込み-----------*/
    //loadFromCSV を使う
    cv::Ptr<cv::ml::TrainData> train_data = cv::ml::TrainData::loadFromCSV("train.csv", 0,-2,0);

    /*-----------学習データの特徴量データの切り出し-----------*/
    //getSamples() を使ってデータを切り出して Mat の行列に代入する
    cv::Mat traindata(4, 2, CV_32FC1);
    traindata = train_data->getSamples();
    cout << "traindata:\n "<< traindata << "\n";
    cout << traindata.rows << " x " << traindata.cols << "\n\n";

    /*-----------学習データのラベルの作成-----------*/
    //学習データ 1 行に対して 1 ラベルが対応する
    //Mat の行列にラベルを代入する
    int labels[4] = { 1, -1, -1, -1 };
    cv::Mat label(4, 1, CV_32SC1, labels);
    cout << "label:\n" << label << "\n";
    cout << label.rows << " x " << label.cols << "\n\n";

    /*-----------SVM 識別器の構築-----------*/
    //SVM::create() を使う
    cv::Ptr<cv::ml::SVM> svm = cv::ml::SVM::create();

    /*-----------SVM のパラメータの設定-----------*/
    svm->setType(cv::ml::SVM::C_SVC);
    svm->setKernel(cv::ml::SVM::LINEAR);
    svm->setTermCriteria(cv::TermCriteria(cv::TermCriteria::MAX_ITER, 100, 1e-6));
```

```
// svm->setC(1.0);//デフォルト値の際は記載しない

    /*-----------SVMの学習-----------*/
    //svm->train()を使う．引数で学習データ行列とデータの格納方向(行方向ならROW_SAMPLE)，ラベル行列を指定する
    svm->train(traindata, cv::ml::ROW_SAMPLE, label);

    /*-----------テストデータの読み込み-----------*/
    //loadFromCSVを使う
    cv::Ptr<cv::ml::TrainData> test_data = cv::ml::TrainData::loadFromCSV("test.csv", 0,-2,0);

    /*-----------テストデータの特徴量データの切り出し-----------*/
    //getSamples()を使ってデータを切り出してMatの行列に代入する
    cv::Mat testdata(2, 2, CV_32FC1);
    testdata = test_data->getSamples();
    cout << "testdata:\n" << testdata << "\n";
    cout << testdata.rows << " x " << testdata.cols << "\n\n";

    /*-----------SVMによる識別器でテストデータのラベル値を予測-----------*/
    //svm->predict()を使う．引数でテストデータの行列とSVMによる予測ラベル行列を指定する．
    cv::Mat response;
    svm->predict(testdata,response);
    cout << "response:\n" << response << "\n";
    cout << response.rows << " x " << response.cols << "\n";

    system("PAUSE");
    return 0;

}

```

実行結果

```
traindata:
 [501, 10;
 255, 10;
 501, 255;
 10, 501]
4 x 2

label:
```

```
[1;
 -1;
 -1;
 -1]
4 x 1

testdata:
[255, 255;
 490, 15]
2 x 2

response:
[-1;
 1]
2 x 1
続行するには何かキーを押してください . . .
```

SVM->train や SVM->predict の引数には Mat 行列を入れているところに注意してください．また，SVM->train の引数に学習データの Mat 行列と学習データのラベルの Mat 行列を入れていますが，正例と負例の学習データを連結した行列 (上記の例だと traindata, label) を引数で指定してください．学習データには 2 次元ベクトルが 4 つあり，それぞれのベクトルにラベルを振るので，labels には 4 つのラベル値 (1,-1,-1,-1) を代入しています．正例と負例の学習データを連結した行列 (上記の例だと traindata，label) を引数で指定することにより，traindata のラベルである label に基づいて正例と負例を判定する識別器を作成することができます．**正例と負例の学習データを連結していない行列を引数で指定するケースが多いので注意してください．ラベルを正しく付けなかったり，正例のみ，負例のみの学習データを引数に指定しても，正例と負例を判定する識別器は作れません．**

学習データ，学習によって求まる識別境界面 (領域) とサポートベクター，テストデータを図 6 に示します．緑の領域が正例，青の領域が負例を表します．黒点が正例の学習データ，白点が負例の学習データ，灰色がかかっている点がサポートベクター，赤点がテストデータです．この図を見てもらうとわかるように，サポートベクターの真ん中に線形の識別境界面が作成されていて，妥当な識別境界面であることがわかります．また，テストデータを見ると，(255,255) が負例で，(490,15) が正例と判定されており，妥当な結果であることがわかります．

また，SVM にはソフトマージンやカーネルなどのパラメータがあります．各パラメータに適当な値を入れて識別器を作成して良い結果を示すこともありますが，パラメータ値を調整することで認識性能が良くなることが良くあります．そのため，識別器を学習する際，パラメータをチューニングする必要があります．各パラメータの値を少しずつ変化させながら識別器を作成し，評価します．そして，精度が高くなるようにパラメータの値を設定します．

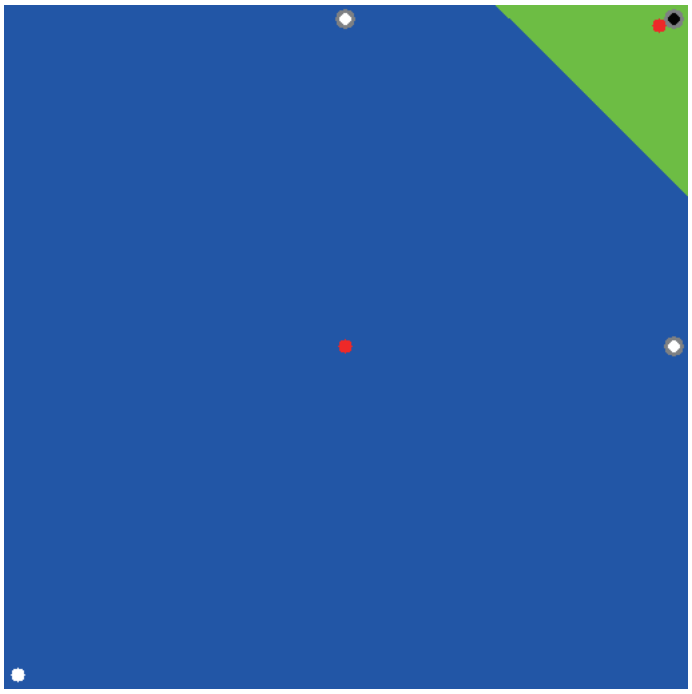

図 6: 特徴空間の様子

4.7 行列の処理

2 つの行列を連結するには cv::vconcat を使うと便利です．下記に vconcat を使用したプログラムコードを示します．行列 A と行列 B を縦方向に連結した行列 C を作成しています．vconcat の引数は Mat 行列であることに注意してください．

プログラムコード

```
1 #include <iostream>
2 #include <opencv2/opencv.hpp>
3 using namespace std;
4
5 int main()
6 {
7     float a[6] = { 1,2,3,4,5,6 };
8     cv::Mat A(3, 2, CV_32FC1,a);
9
10    cout << "A=" << A << "\n";
11    cout << A.rows << " x " << A.cols << "\n\n";
12
13    float b[8] = { 7,8,9,10,11,12,13,14 };
14    cv::Mat B(4, 2, CV_32FC1, b);
15
```

```
    cout << "B=" << B << "\n" ;
    cout << B.rows << " x " << B.cols << "\n\n";

    cv::Mat C(7, 2, CV_32FC1);

    cv::vconcat(A, B, C);

    cout << "C=" << C << "\n";
    cout << C.rows << " x " << C.cols << "\n\n";

    system("PAUSE");
    return 0;
}
```

実行結果

```
A=[1, 2;
 3, 4;
 5, 6]
3 x 2

B=[7, 8;
 9, 10;
 11, 12;
 13, 14]
4 x 2

C=[1, 2;
 3, 4;
 5, 6;
 7, 8;
 9, 10;
 11, 12;
 13, 14]
7 x 2

続行するには何かキーを押してください . . .
```

5 参考文献

本資料をまとめるにあたり下記の文献を参考にした．機械学習に関する書籍は多く出版されているので，興味のある人はさらに勉強するとよい．

- 石井健一郎, 前田英作, 上田修功, 村瀬洋, “わかりやすいパターン認識,” オーム社
- 荒木雅弘, “フリーソフトではじめる機械学習入門,” 森北出版株式会社
- https://docs.opencv.org/4.2.0/

課題 5

以下の実験を行い，そのレポートを提出せよ．特に理由のない限り，IED の Linux 環境で g++のコンパイラ，OpenCV4.2.0 を用いて実験を行うこと．課題において，プログラムを実行する際のコマンドラインの例が示されている場合は，その例に倣ったプログラムを作成すること．各課題について，回答に至った考え方，作成にあたり工夫した点を記載すること．プログラムの動作出力結果 (ターミナルのスクリーンショット，課題に引数の例が示されている場合は全ての引数の例を試した出力結果もスクリーンショットに含めてください) をレポートに含めてください．その際には作成したソースコードの仕様（コンパイル方法，プログラムの実行方法，実行時に入力する引数の説明）の説明もつけてください．なお，実験資料の Web ページに OpenCV4.2.0 のマニュアルページへのリンクも付けている．下記の課題を取り組むにあたり，本実験資料に加えて，マニュアルページも参考にすること．また，レポートをメールで提出する際，添付ファイルのサイズには十分に注意すること．

0. (実験準備) 実験資料の Web ページにある `sample.cpp` を取得せよ (`sample.cpp` は `sample.zip` の中にある)．このソースをコンパイルして正しく実行できることを確かめよ．この課題は OpenCV の動作がちゃんと行えているかを確認するものであり，提出する必要はない．

1. 以下の各問題の答えを求めるプログラムを作成し，計算結果を示すこと．

   (1) 行列 A を $\begin{pmatrix} 1 & -3 & 5 \\ 3 & 2 & 4 \\ 6 & 4 & 3 \end{pmatrix}$ としたとき，行列 $A + A^t$ を求めよ．転置は*.t で求める．

   (2) 行列 $\begin{pmatrix} 3 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 3 \end{pmatrix}$ の行列式を計算せよ．行列式は cv::determinant を用いる

   (3) 行列 $\begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}$ の逆行列を計算せよ．逆行列は*.inv で求める．

2. 実験資料の Web ページにある `binary2026.bmp` を取得せよ (`binary2026.bmp` は `binary2026.zip` の中にある)．

   (1) この画像をグレースケール変換してから，閾値処理により 2 値化を行うプログラムを作成し，閾値をいろいろと変えた時の処理画像を表示せよ．レポートには，閾値を変えた時の処理画像の数例を示すこと．閾値処理は，各画素の画素値と閾値とを比較して，その大小により白 (画素値:255) または黒 (画素値:0) に変換するものであり，OpenCV の関数で実装すること．実行コマンドは閾値を x とすると 「$./2410456_kadai5-2 binary2026.bmp x」で，x=100 の時，「$./2410456_kadai5-2 binary2026.bmp 100」．

   (2) 閾値を変化させることで処理画像がどのように変化するかを答えよ．

3. 実験資料の Web ページにある distribution2026.zip を取得せよ．この中には 2 クラス分類するためのデータが入っている．class1tr_2026.csv と class2tr_2026.csv はクラス 1，クラス 2 の学習データ，class1te_2026.csv と class2te_2026.csv はクラス 1，クラス 2 のテストデータである．ファイルの見方としては，列方向が特徴量の値，行方向がサンプルになる．つまり，学習データは各クラス 200 サンプル，テストデータは各クラス 50 サンプルある．

   (1) クラス 1，クラス 2 の学習データを用いて，クラス 1，クラス 2 を判定する識別器を作成し，テストデータを用いて，それぞれのデータが正しくクラス分類できるかを評価 (テストデータのサンプルがどのくらいの割合で正しく分類できたか) せよ．実行コマンドは，「$./2410456_kadai5-3 class1tr_2026.csv class2tr_2026.csv class1te_2026.csv class2te_2026.csv」．学習アルゴリズムは，線形カーネルによる SVM を用いること．

   (2) 分類結果について考察してください．

4. (発展課題) マルウェアを検知するための識別器を作成することとした．そのため，まず学習データを作成した．仮想環境においてアプリケーションを実行したときの単位時間あたりに呼び出されたシステムコール A の出現頻度の平均値とシステムコール B の出現頻度の平均値を取得し，識別器で用いる特徴量とする．良性のアプリケーション，マルウェアを 1 アプリケーションずつ実行し，データを取得した．実験資料の Web ページにある detection2026.zip を取得せよ．この中には，良性アプリケーションを実行したときのデータ (benign.csv) とマルウェアを実行したときのデータ (malware.csv) が入っている (もちろん，実際は違うがそう思おう)．このファイルには，それぞれ単位時間当たりのシステムコール A の出現頻度の平均値とシステムコール B の出現頻度の平均値が記載されている (列方向が 2 つの特徴量で，行方向がアプリケーションとする，1 行が 1 アプリケーションのデータで行数分のアプリケーションの実行結果があるとする)．そして，後日，作成した識別器の性能を評価するために，同様の仮想環境でデータを取得しファイル (benign_te.csv,malware_te.csv) に保存した．benign_te.csv は，良性のアプリケーションを実行した際のデータ，malware_te.csv はマルウェアを実行した際のデータである (列方向が 2 つの特徴量で，行方向がアプリケーションとする，1 行が 1 アプリケーションのデータで行数分のアプリケーションの実行結果があるとする)(もちろん，実際は違うがそう思おう)．benign.csv と malware.csv の 2 つファイルを使って，マルウェアを検知するための識別器を構成し，benign_te.csv,malware_te.csv を使って良性かマルウェアを判定してください．実行コマンドは,「$./2410456_kadai5-4 detection2026/」．

   (1) benign.csv と malware.csv の 2 つファイルを使って線形カーネルによる SVM を用いた識別器をプログラムで実装せよ．そして，benign_te.csv,malware_te.csv をアプリケーションごと (ファイルの 1 行ごと) に良性，マルウェアを判定をせよ．

   (2) benign.csv と malware.csv の 2 つファイルを使って RBF カーネル，多項式カーネルによる SVM を用いた識別器をプログラムで実装せよ．そして，benign_te.csv,

malware_te.csv をアプリケーションごと (ファイルの 1 行ごと) に良性，マルウェアを判定をせよ．2 つのカーネルそれぞれについてパラメータチューニングを行い，パラメータの値と識別率の関係を表すグラフを示し，一番判定結果が良かった結果を報告してください．

(3) (1) と (2) の判定精度を比較し，結果について考察せよ．

5. (発展課題) 実験資料の Web ページにある mail2026.zip を取得せよ．この中には，送られてきたメールの文章から特徴を抽出したデータが入っている．Nmail2026.csv は，通常のやり取りをした際のメールから特徴を抽出したデータで，Amail2026.csv は，スパムメールから特徴を抽出したデータである (もちろん，実際は違うがそう思おう)．また，後日収集した Nmail_t2026.csv，Amail_t2026.csv がある (もちろん，実際は違うがそう思おう)．それぞれのファイルは，列方向が特徴量の値で，行方向がメールの件数となる．Nmail2026.csv と Amail2026.csv の 2 つのファイルを使って，スパムメールフィルタの機能を持つ識別器をプログラムで実装してください．

(1) ベイズの定理を説明せよ．文献を調べた場合は，利用した参考文献を明示せよ．

(2) Nmail2026.csv と Amail2026.csv のそれぞれ最初の 5 行のデータを用いて，ベイズの定理による識別器をプログラムで実装してください．そして，Nmail_t2026.csv，Amail_t2026.csv のデータに対して，作成した識別器でスパム判定を行い，その正解率を求めよ．実行コマンドは，引数で学習データの何行目まで使うかを指定するものとし，例えば 5 行目まで使うとすると，「$./2410456_kadai5-5 mail2026/ 5」．

(3) (2) の実験で，

- 最初の 10 行を使って識別器を学習
- 最初の 50 行を使って識別器を学習
- 最初の 100 行を使って識別器を学習
- 最初の 200 行を使って識別器を学習

を行い，スパム判定の正解率の変化を確認し，その結果について考察せよ．

6. (発展課題) 実験資料の Web ページにある ball2026.zip を取得せよ．このファイルの中には，3 種類のボールの画像が入っている．各ボールに対して 3 枚の画像がある．これらの画像を学習データとしてボールを識別する識別器を作成せよ．実行コマンドは，「$./2410456_kadai5-6 ball2026/」．また，unknown 画像が 3 枚入っている．作成した識別器を利用してどのボールかを識別し，正しく識別されたかを報告し，結果について考察せよ．画像特徴量，機械学習アルゴリズムは各自が選択するものとする．